



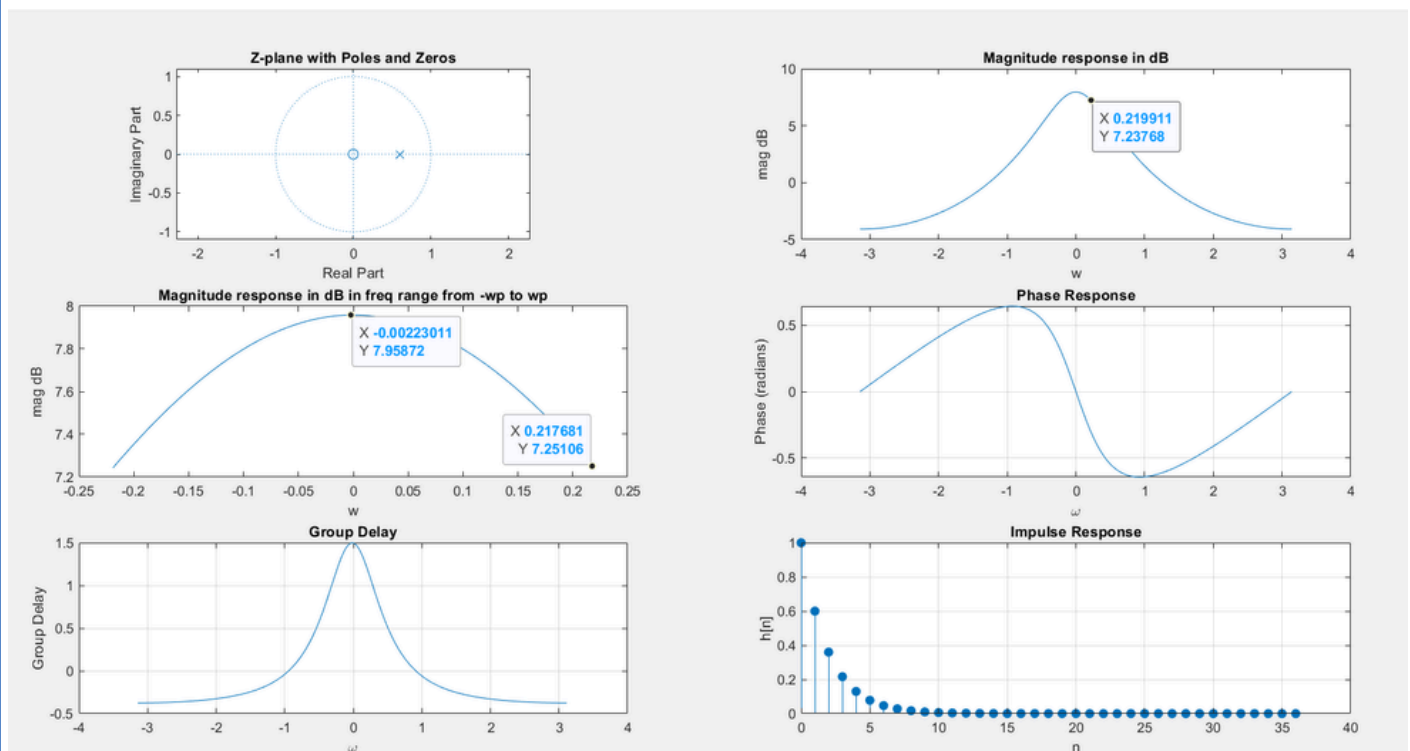
ECE354s: DSP Project

Nada Ashraf Mahmoud	2300629
Tasneem Aaref Mahrous	2300393
Omar Sherif Taha	2300213
Alaa Atef El-Sayed	2300377
Hazem Saber Hanafi	2300144

1.First Order Discrete Time Low Pass Filter

$$H(z) = \frac{1}{1 - a z^{-1}}$$

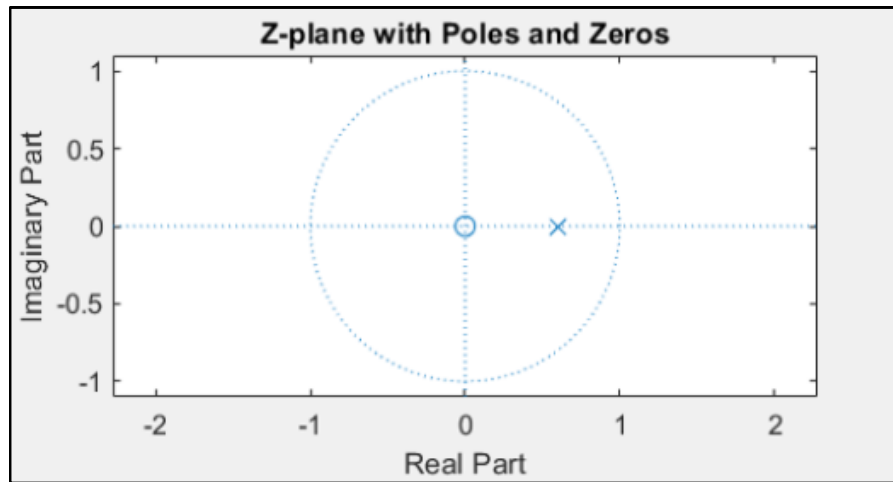
for $a = 0.6$



- plotting zeros and poles of transfer function:
first we the coefficients of the numerator and denominator of the transfer function and zplane function is used to plot the graph

```
% plotting zeros and poles of transfer function
A= 0.6;
a = 1;
b = [1 -A];

subplot(3,2,1)
zplane(a,b);
title('Z-plane with Poles and Zeros');
```



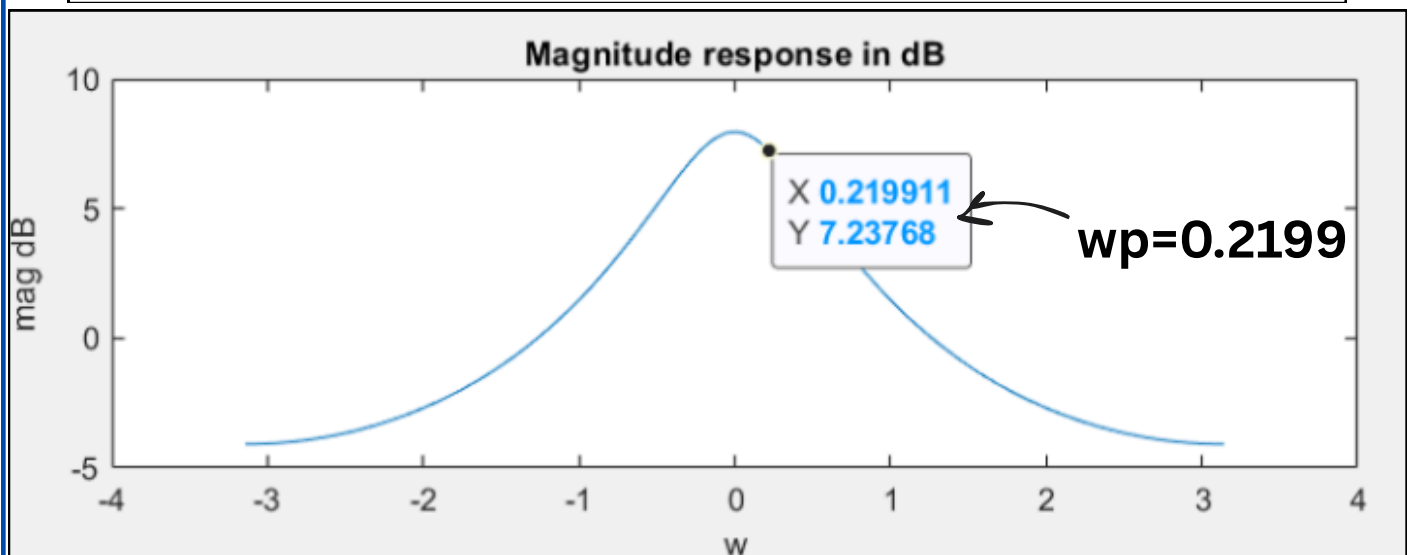
- Magnitude response of the filter in dB in freq. range from $-\pi$ to π :

to determine the magnitude of the transfer function I substitute by $e^{j\omega}$ instead of z

The code:

```
% Magnitude response in dB in freq range from -pi to pi
w = -pi:pi/100:pi;
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);
mag_dB = 20*log10(mag);

subplot(3,2,2)
grid on;
plot (w,mag_dB);
xlabel ('w');
ylabel ('mag_dB');
title('Magnitude response in dB');
```

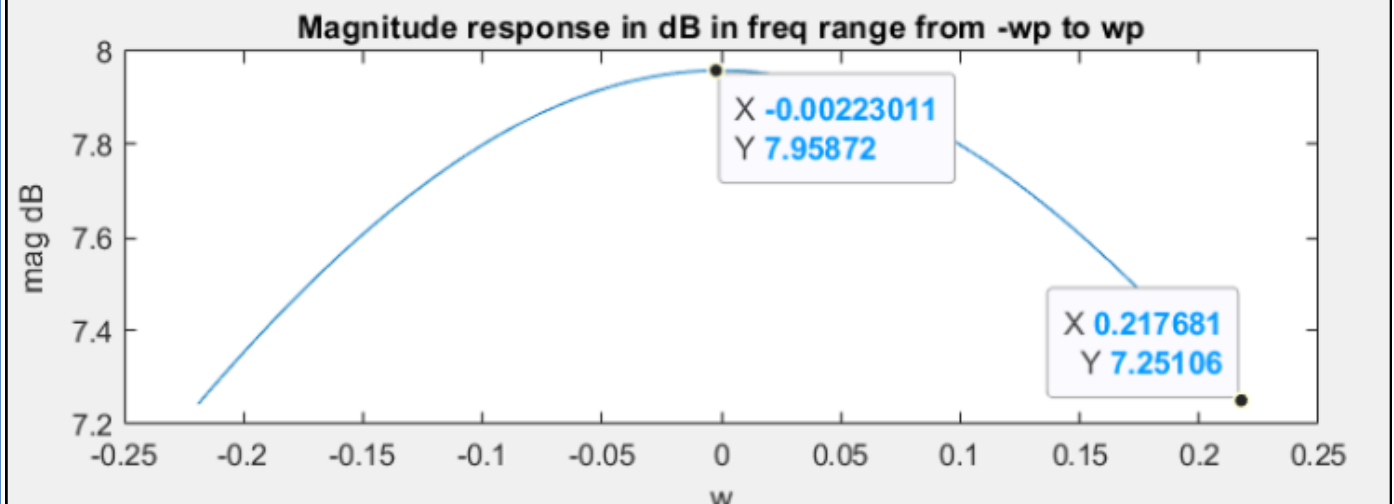


from the mag. response graph we can observe it is a LPF as it has higher mag. response around Low frequencies and lower mag. response around the high frequencies

- Magnitude response in dB in freq range from $-\omega_p$ to ω_p to calculate the passband ripples

```
%% Magnitude response in dB in freq range from  $-\omega_p$  to  $\omega_p$  & evaluating the ripples
```

```
w = -0.2199:pi/1000:0.2199;  
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);  
mag_dB = 20*log10(mag);  
  
subplot(3,2,3)  
grid on;  
plot (w,mag_dB);  
xlabel ('w');  
ylabel ('mag_dB');  
title('Magnitude response in dB in freq range from  $\omega_p$  to  $\pi$ ');
```

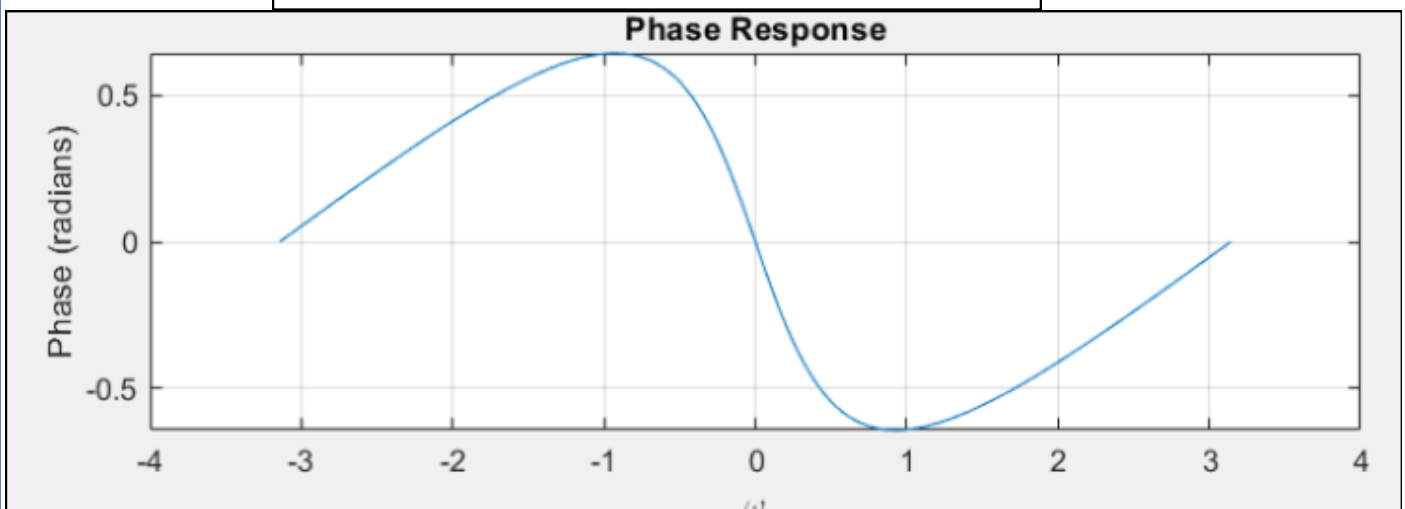


from the graph:

the ripples = max value - min. value = 7.958 - 7.251 = 0.707 dB

- Phase response of the system:

```
%% Phase response
w = -pi:pi/100:pi;
real_part = (1 - A*cos(w));
imag_part = (A*sin(w));
phase = -atan2(imag_part, real_part);
subplot(3,2,4)
plot(w, phase);
grid on;
xlabel('\omega');
ylabel('Phase (radians)');
title('Phase Response');
```



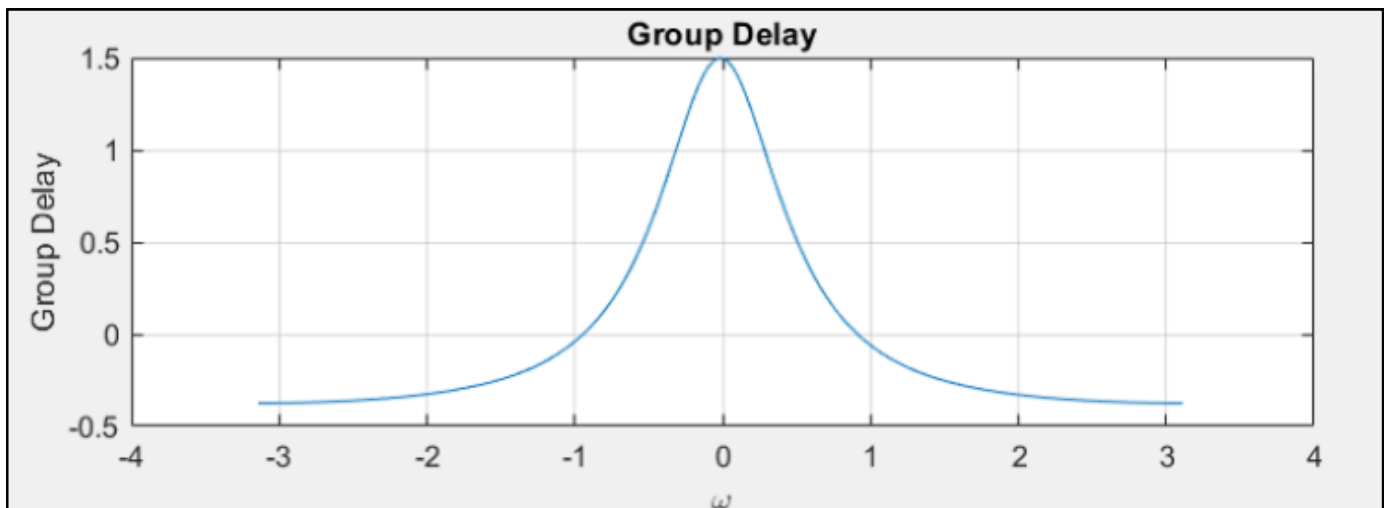
The phase response varies continuously with frequency and exhibits phase wrapping effects, which cause the phase plot to appear as alternating swings.

- group delay:

```
% numerical derivative
dphi = diff(phase);
dw = diff(w);

gd = -dphi ./ dw;

% plotting
subplot(3,2,5)
plot(w(1:end-1), gd);
grid on;
xlabel('\omega');
ylabel('Group Delay');
title('Group Delay');
```



The group delay is relatively high at low frequencies and gradually decreases as the frequency increases. This indicates that the low-frequency components of the signal experience larger delays compared to the high-frequency components. Such behavior reflects the frequency-dependent phase characteristics of the filter and is commonly observed in digital low-pass filters with non-linear phase responses.

- **calculation of inverse z-transform of the transfer function**

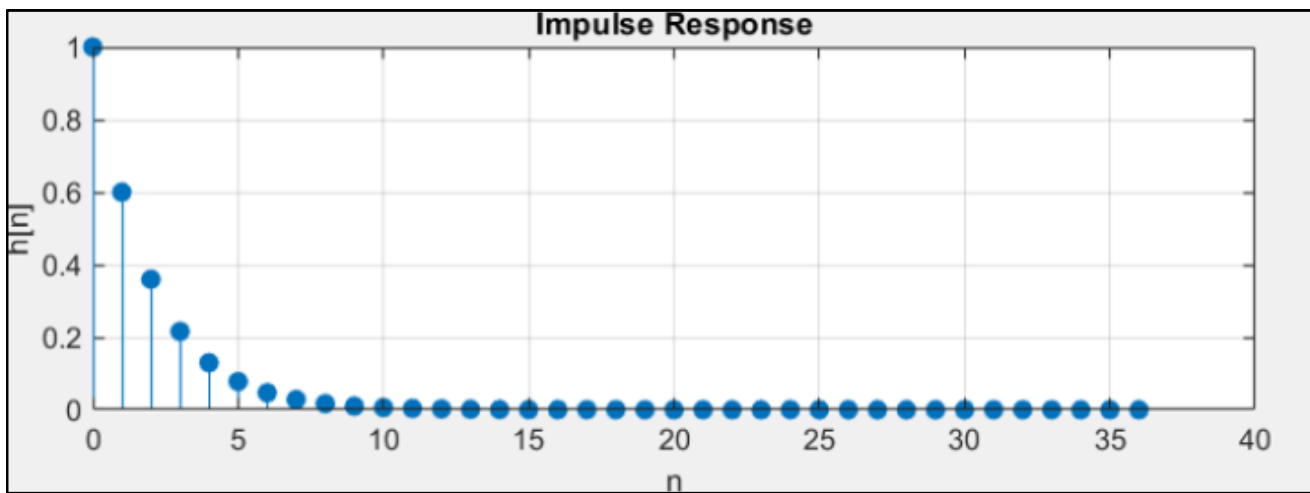
```
%% calculation of inverse z-transform
```

```
syms z n
H_z = 1/(1 - A*z^(-1));
h_n = iztrans(H_z, z, n);
```

```
%% Impulse response plot
```

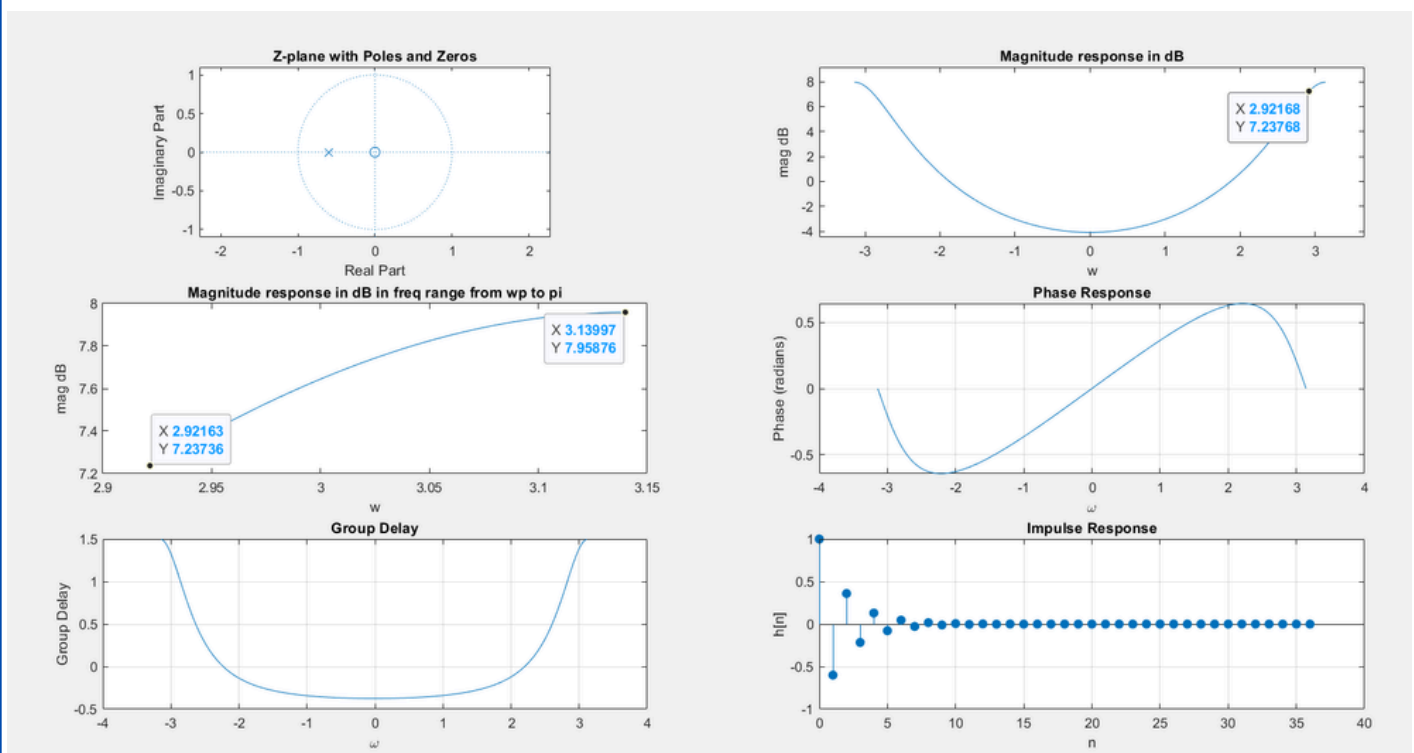
```
n_plot = 0:36;
h_plot = double(subs(h_n, n, n_plot));
```

```
subplot(3,2,6)
stem(n_plot,h_plot,'filled');
grid on;
xlabel('n');
ylabel('h[n]');
title('Impulse Response');
```



The impulse response has decreasing amplitude and eventually converges to zero, indicating a stable system.

For $a = -0.6$



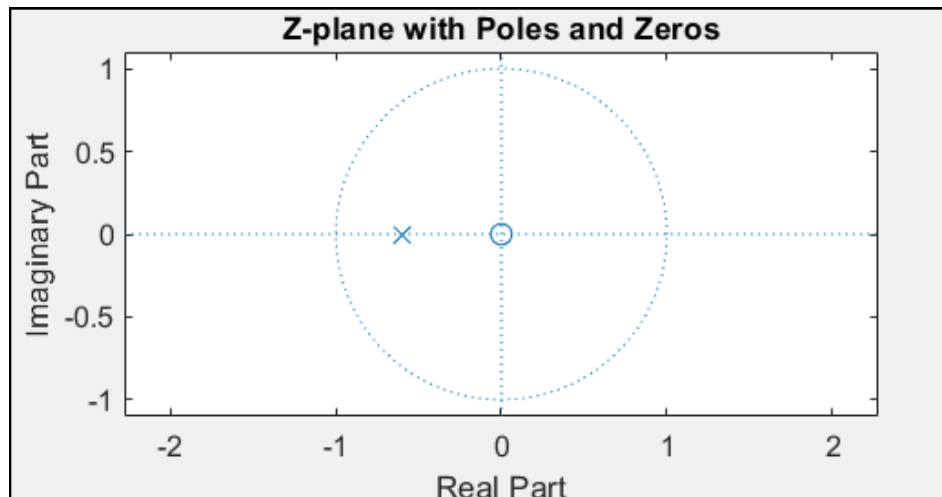
- plotting zeros and poles of transfer function:
first we the coefficients of the numerator and denominator of the transfer function and zplane function is used to plot the graph

```

%% plotting zeros and poles of transfer function
A= 0.6;
a = 1;
b = [1 -A];

subplot(3,2,1)
zplane(a,b);
title('Z-plane with Poles and Zeros');

```



- Magnitude response of the filter in dB in freq. range from $-\pi$ to π :

to determine the magnitude of the transfer function I substitute by $e^{-j\omega}$ instead of z

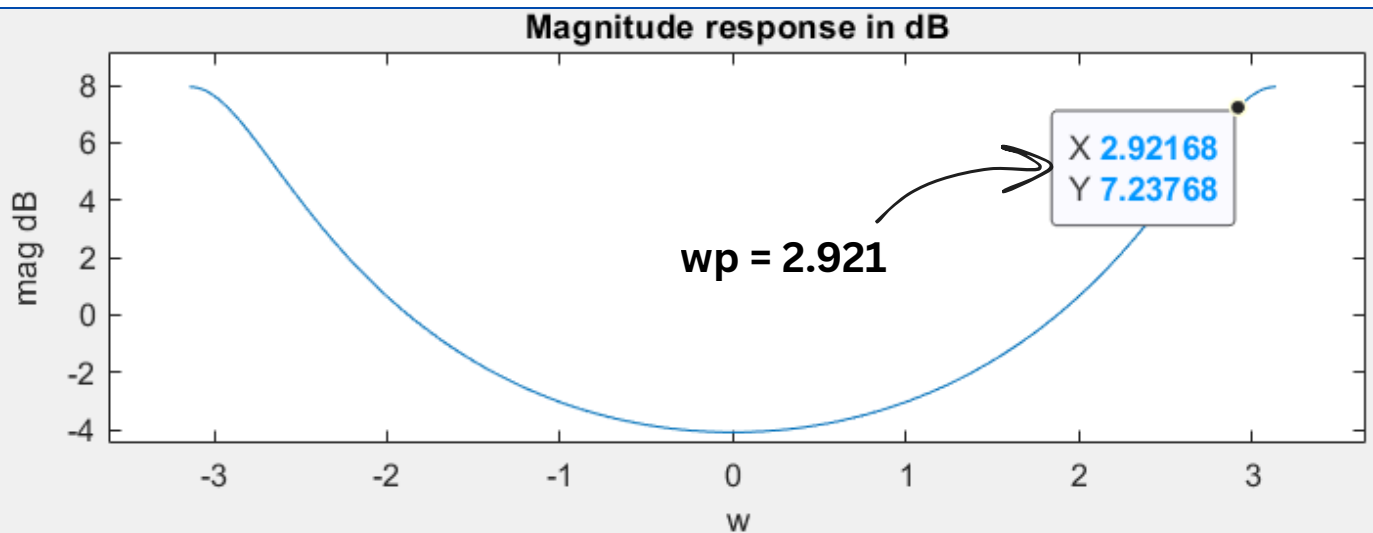
The code:

```

%% Magnitude response in dB in freq range from -pi to pi
w = -pi:pi/100:pi;
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);
mag_dB = 20*log10(mag);

subplot(3,2,2)
grid on;
plot (w,mag_dB);
xlabel ('w');
ylabel ('mag_ dB');
title('Magnitude response in dB');

```

from the mag. response we can observe that this is a HPF

- Magnitude response in dB in freq range from w_p to π to calculate the ripples

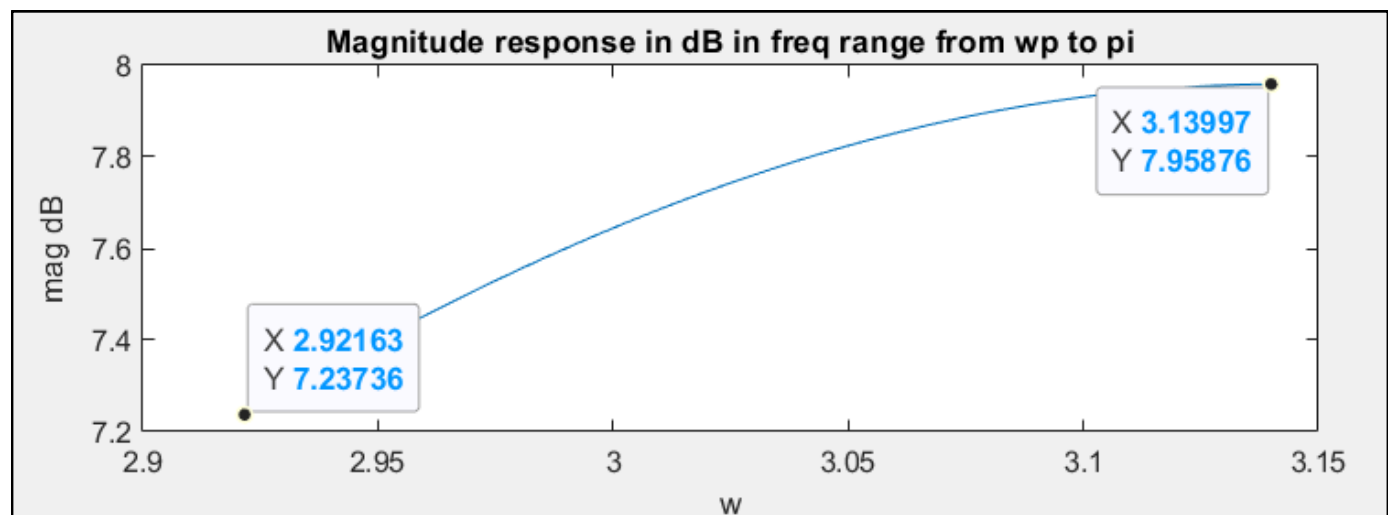
this filter represents a highpass filter so the passband region will be from w_p to π not from $-w_p$ to w_p !

The code:

%% Magnitude response in dB in freq range from w_p to π & evaluating the ripples

```
w = 2.91:pi/1000:pi;
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);
mag_dB = 20*log10(mag);

subplot(3,2,3)
grid on;
plot (w,mag_dB);
xlabel ('w');
ylabel ('mag_ dB');
title('Magnitude response in dB in freq range from  $w_p$  to  $\pi$ ');
```



from the graph:

the ripples = max value - min. value = $7.958 - 7.237 = 0.721$ dB

- Phase response of the system:

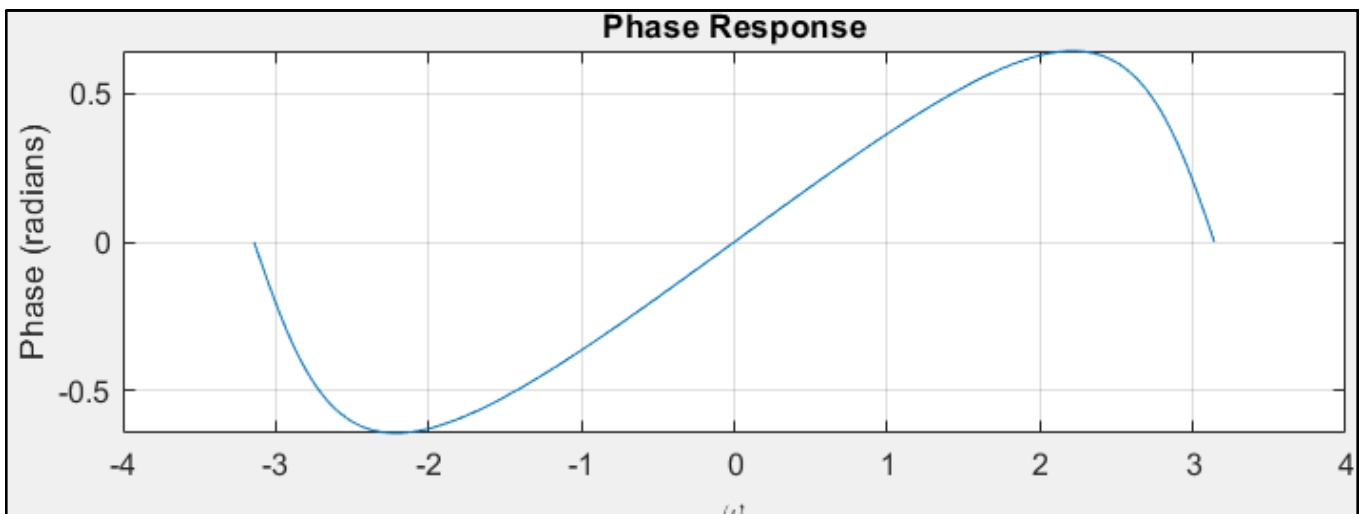
```
%% Phase response

w = -pi:pi/100:pi;

real_part = (1 - A*cos(w));
imag_part = (A*sin(w));

phase = -atan2(imag_part, real_part);

subplot(3,2,4)
plot(w, phase);
grid on;
xlabel('\omega');
ylabel('Phase (radians)');
title('Phase Response');
```



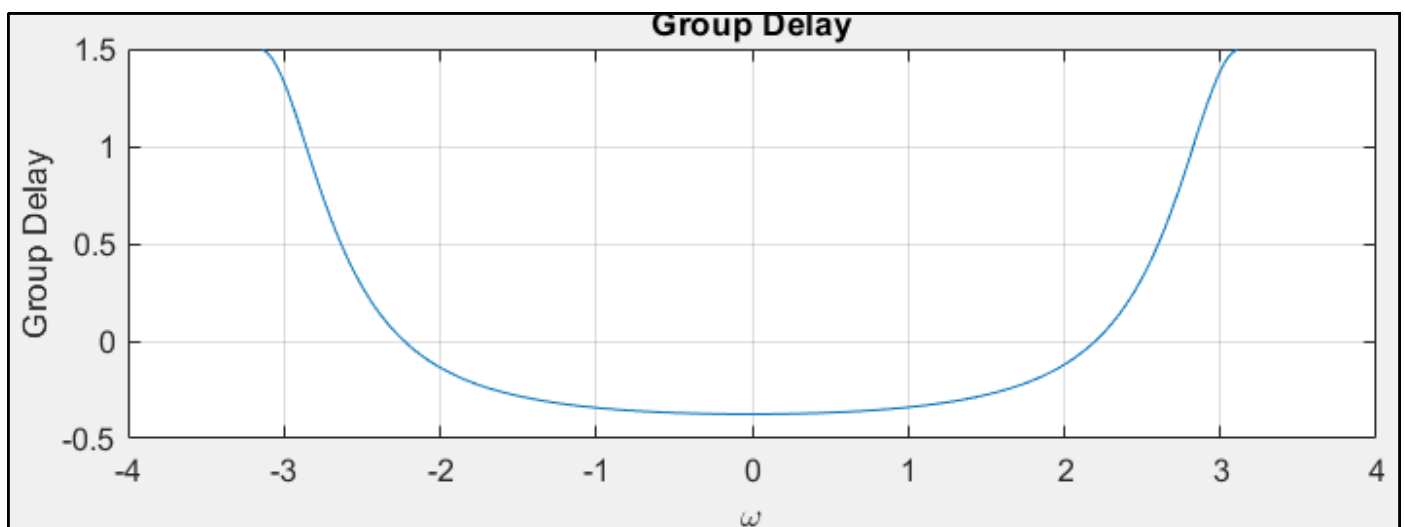
The phase response is smooth but nonlinear, indicating frequency-dependent phase shifts. This is a typical behavior of first-order IIR high-pass filters and may lead to phase distortion near the cutoff frequency.

- group delay:

```
% numerical derivative
dphi = diff(phase);
dw = diff(w);

gd = -dphi ./ dw;

% plotting
subplot(3,2,5)
plot(w(1:end-1), gd);
grid on;
xlabel('\omega');
ylabel('Group Delay');
title('Group Delay');
```



The group delay increases at higher frequencies and reaches its maximum values within the passband of the high-pass filter. This means that the transmitted high-frequency components experience larger delays compared to low-frequency components. The frequency-dependent nature of the group delay indicates that the filter does not possess a perfectly linear phase response, which may introduce phase distortion.

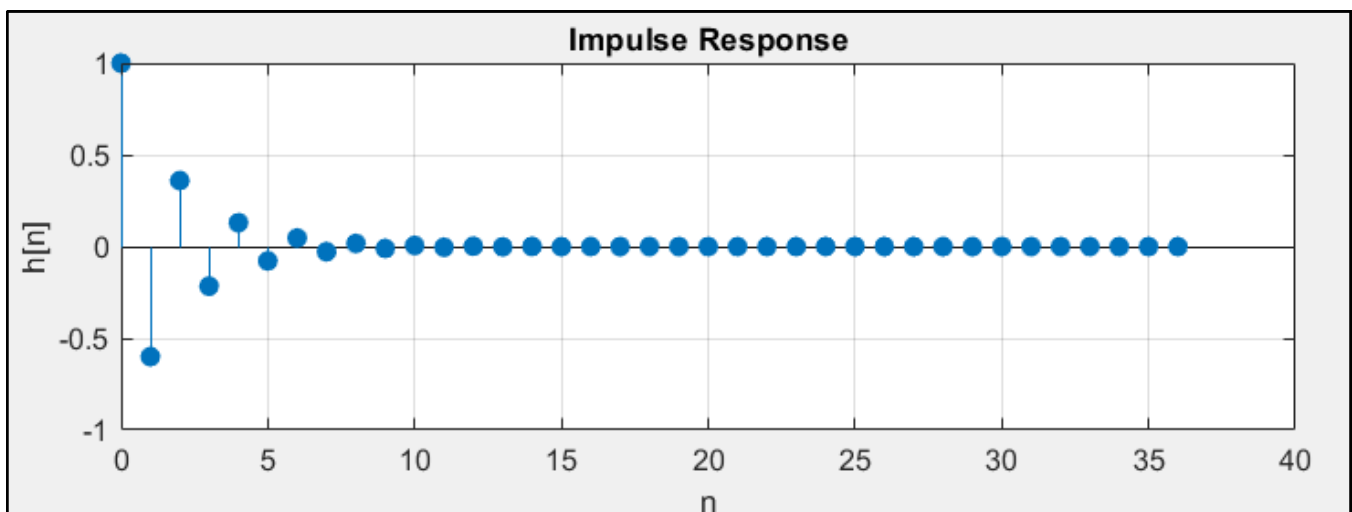
- calculation of inverse z-transform of the transfer function

```
%% calculation of inverse z-transform
```

```
syms z n  
H_z = 1/(1 - A*z^(-1));  
h_n = iztrans(H_z, z, n);
```

```
%% Impulse response plot
```

```
n_plot = 0:36;  
h_plot = double(subs(h_n, n, n_plot));  
  
subplot(3,2,6)  
stem(n_plot, h_plot, 'filled');  
grid on;  
xlabel('n');  
ylabel('h[n]');  
title('Impulse Response');
```



The impulse response oscillates with decreasing amplitude and eventually converges to zero, indicating a stable system. The alternating sign of the samples results from the negative pole of the filter.

```

function system_analysis(num, den, wp)
    w = linspace(-pi, pi, 1024);
    figure('Name', 'Filter Analysis Dashboard', 'Position', [100, 100, 1200, 800]);
    H = freqz(num, den, w);
    mag_dB = 20*log10(abs(H));
    % 1. Pole-Zero Plot
    subplot(2, 3, 1);
    zplane(num, den);
    grid on;
    title('Pole-Zero Map');
    % 2. Full Magnitude Response (-pi to pi)
    subplot(2, 3, 2);
    plot(w, mag_dB);
    grid on;
    xlim([-pi, pi]);
    xlabel('\omega (radians)');
    ylabel('Magnitude (dB)');
    title('Magnitude Response (-\pi \rightarrow \pi)');
    % 3. Passband Magnitude Response (-wp to wp) to check ripples
    subplot(2, 3, 3);
    plot(w, mag_dB);
    grid on;
    xlim([-wp, wp]);
    xlabel('\omega (radians)');
    ylabel('Magnitude (dB)');
    title('Magnitude Response (-\omega_p \rightarrow \omega_p)');
    % 4. Phase Response
    subplot(2, 3, 4);
    phi = angle(H);
    plot(w, phi);
    grid on;
    xlim([-pi, pi]);
    xlabel('\omega (radians)');
    ylabel('Phase (radians)');
    title('Phase Response');
    % 5. Group Delay
    subplot(2, 3, 5);
    gd = grpdelay(num, den, w);
    plot(w, gd);
    grid on;
    xlim([-pi, pi]);
    xlabel('\omega (radians)');
    ylabel('Group Delay (samples)');
    title('Group Delay');
    % 6. Impulse Response
    subplot(2, 3, 6);
    [h, t] = impz(num, den, 50);
    stem(t, h);
    grid on;
    xlabel('n');
    ylabel('h[n]');
    title('Impulse Response');
end

```

This function is a general that allows us to study the behavior of any filter we made by providing its numerator coefficients, denominator coefficients, and passband edge frequency as inputs to the function. First, it creates a frequency vector from $-\pi$ to π and uses (freqz) to calculate the filter's frequency response $H(ej\omega)$. The response magnitude is converted to decibels to make attenuation and gain easier to observe. The function then displays 6 plots in one subplot: a pole-zero map to check stability, magnitude response, passband magnitude response from $(-\omega_p$ to $\omega_p)$ to inspect passband ripples, a phase response to show the phase shift at each frequency and see the phase linearity, a group delay plot to measure how much different frequency components are delayed, and an impulse response to see the filter's behavior in the time domain.

2. Third order filter

Parameters:

```
wc = 0.25 * pi;           % Cutoff frequency
width = 0.1 * pi;         % Transition band width
ws = wc + (width / 2);    % Stopband edge
wp = wc - (width / 2);    % Passband edge
```

```
for r = 0 : 0.001 : 1
    Counter = Counter + 1;
    pole2 = r*exp(j*wp);
    pole3 = r*exp(-j*wp);
    poles = poly([pole1 pole2 pole3]);

    H = freqz(zeros_poly,poles,W);
    Mag_dB = 20*log10(abs(H));
    mag(Counter) = r;
    Storage(Counter) = max(Mag_dB) - min(Mag_dB);

end
[minR,i] = min(Storage);
optimal_r = mag(i);
fprintf('minimum ripples: %.3f dB\n', minR);
fprintf('optimal r: %.3f\n', optimal_r);
```

To meet the requirement of keeping the passband ripple below 1 dB, I tested different values of the pole radius r from 0 to 1 in small steps. For each value of r , I placed a pair of complex conjugate poles at the fixed angle ω_p I got from parameters, built the filter denominator, and calculated the frequency response using `freqz`. Then I measured the ripple in the passband by finding the difference between the maximum and minimum magnitude in dB. I stored this ripple value for every r . After checking all values, I picked the radius that gave the smallest ripple. This way, I automatically found the best pole radius that makes the filter response as smooth as possible in the passband.

Output:

```
minimum ripples: 1.110 dB  
optimal r: 0.888
```

but there was a problem the output ripples was not less than 1 dB, so that doesn't meet the specifications, so we made a modification on the zeros magnitude and we started to change the magnitude of them till we met specifications .

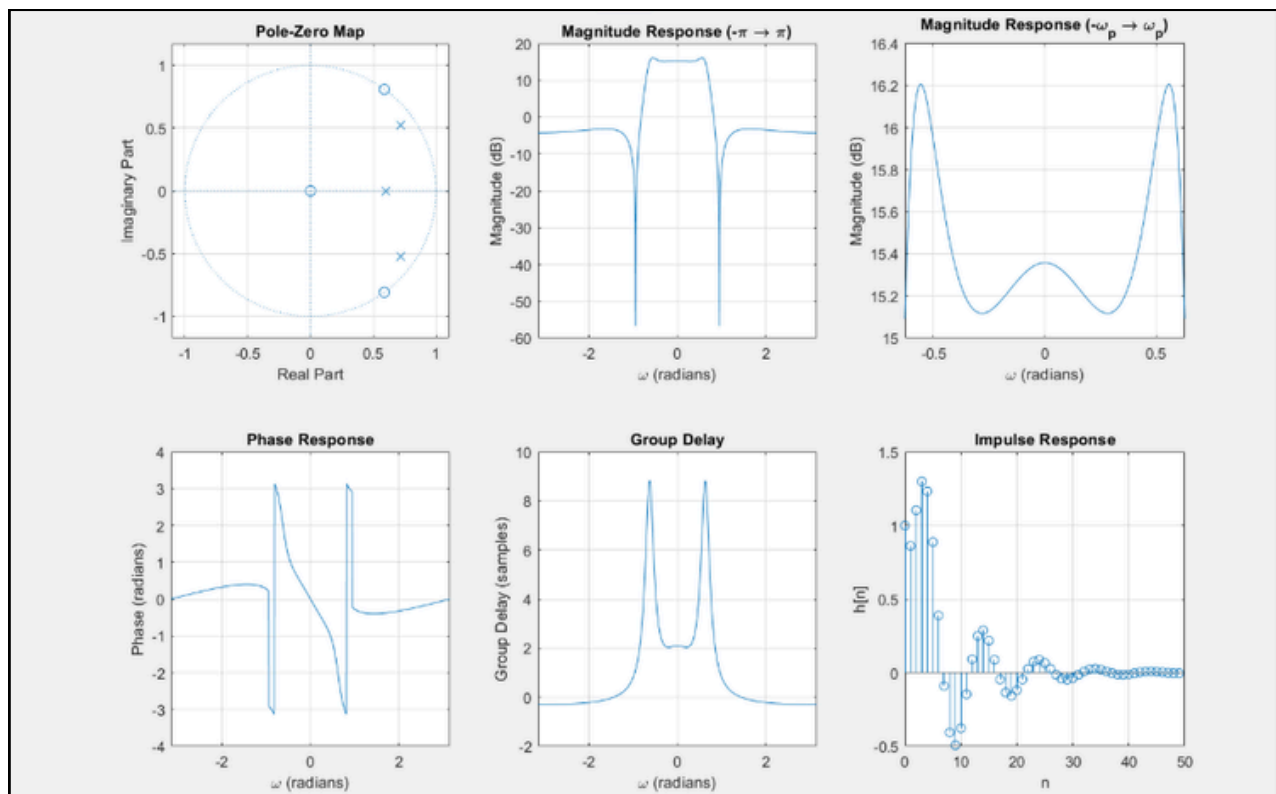
```
%% zeros  
zero1 = 0.3*exp(j*ws);  
zero2 = 0.3*exp(-j*ws);
```

Output:

minimum ripples: 0.648 dB

optimal r: 0.787

Plots:



These plots show the analysis of a 3rd-order low-pass filter. From the pole-zero map, we can see that all poles are inside the unit circle, so the filter is stable. The magnitude response shows that low frequencies pass with almost constant gain, while high frequencies are reduced, which means that is a low-pass filter. In the passband zoom, there is a small ripple in the gain. The phase response is not linear, so the filter introduces phase distortion. The group delay is also not constant at all frequencies, so different frequencies are delayed differently. The impulse response exhibits damped oscillations due to the presence of a complex-conjugate pole pair. These poles introduce a sinusoidal component in the response, while their magnitude being less than one causes the oscillations to gradually decay with time.

2. Fifth Order Filter

Parameters:

```
wp2 = wp/2;  
ws2 = (pi + ws) / 2;
```

```
pole2 = optimal_r * exp(j * wp);  
pole3 = optimal_r * exp(-j * wp);  
pole4 = optimal_r * exp(j * wp2);  
pole5 = optimal_r * exp(-j * wp2);  
poles = poly([pole1 pole2 pole3 pole4 pole5]);  
  
zero3 = exp(j * ws2);  
zero4 = exp(-j * ws2);  
zeros_poly = poly([zero1 zero2 zero3 zero4]);
```

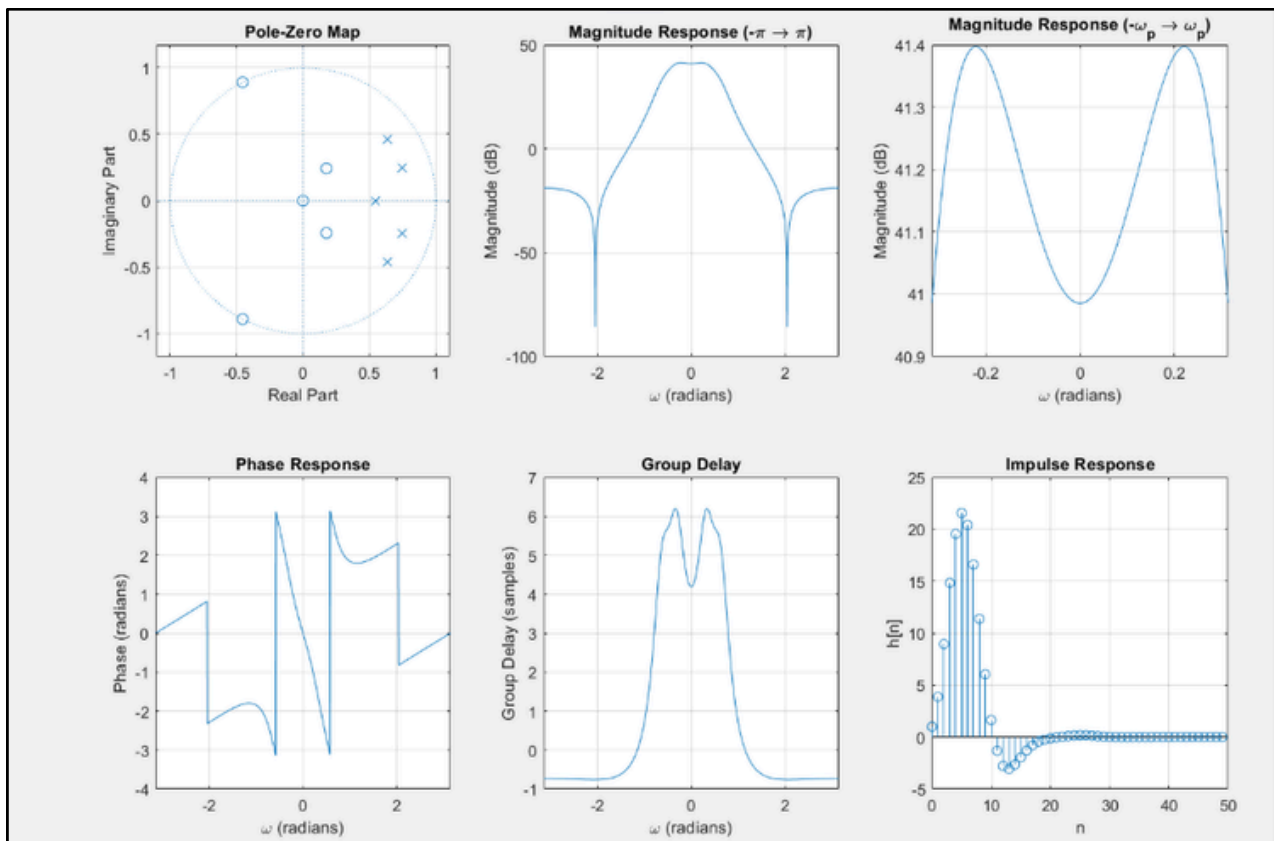
I used the same radius i got from the first part and then instead i sweep on the first pole magnitude.

```
for r = 0 : 0.001 : 1  
    Counter = Counter + 1;  
    pole1 = r;  
    poles = poly([pole1 pole2 pole3 pole4 pole5]);  
    H = freqz(zeros_poly, poles, W);  
    Mag_dB = 20*log10(abs(H));  
    mag2(Counter) = r;  
    Storage2(Counter) = max(Mag_dB) - min(Mag_dB);  
end
```

Output:

```
minimum ripples: 0.412 dB  
optimal r: 0.548
```

Plots:



These plots show the analysis of a 5th-order low-pass filter. From the pole-zero map, all poles are located inside the unit circle, confirming that the filter is stable. Compared to the 3rd-order filter, the magnitude response exhibits a much sharper transition between the passband and stopband, providing better attenuation of high-frequency components. The passband gain remains nearly constant with a small ripple that satisfies the design specs. The phase response becomes more nonlinear due to the increased filter order, resulting in greater phase distortion. The group delay changes near the cutoff frequency, so different frequencies are delayed differently. The impulse response decays with time and shows more ringing than the 3rd-order filter due to the higher filter order. The 5th-order filter has a sharper transition region and better stopband attenuation, but it also introduces more phase distortion and delay variation. Overall, it gives better filtering performance at the expense of phase linearity.

7. Frequency Transformation using Pole-Zero Pattern Rotation:

- HPF

```
84      [z_lpf, p_lpf, k_lpf] = tf2zpk(num, den);
```

The tf2zpk (Transfer Function to Zero-Pole-Gain)function converts the transfer function from its numerator-denominator form into a zero-pole-gain representation. This representation is more convenient for frequency transformations because the poles and zeros can be directly rotated in the z-plane to obtain the new filter(High-Pass and Band-Pass filters).

```
86      z_hpf = z_lpf * exp(1j * pi); %rotating by  $\pi$ 
87      p_hpf = p_lpf * exp(1j * pi);
```

The poles and zeros of the low-pass filter were rotated by π radians in the z-plane by multiplying their locations by ($e^{j\pi}=-1$).

This operation moves every pole and zero to the opposite side of the unit circle while preserving their distances from the origin. As a result, the pole-zero configuration is transformed from that of a low-pass filter to a high-pass filter.

```
89      [num_hpf, den_hpf] = zp2tf(z_hpf, p_hpf, k_lpf);
90      num_hpf = real(num_hpf);
91      den_hpf = real(den_hpf);
```

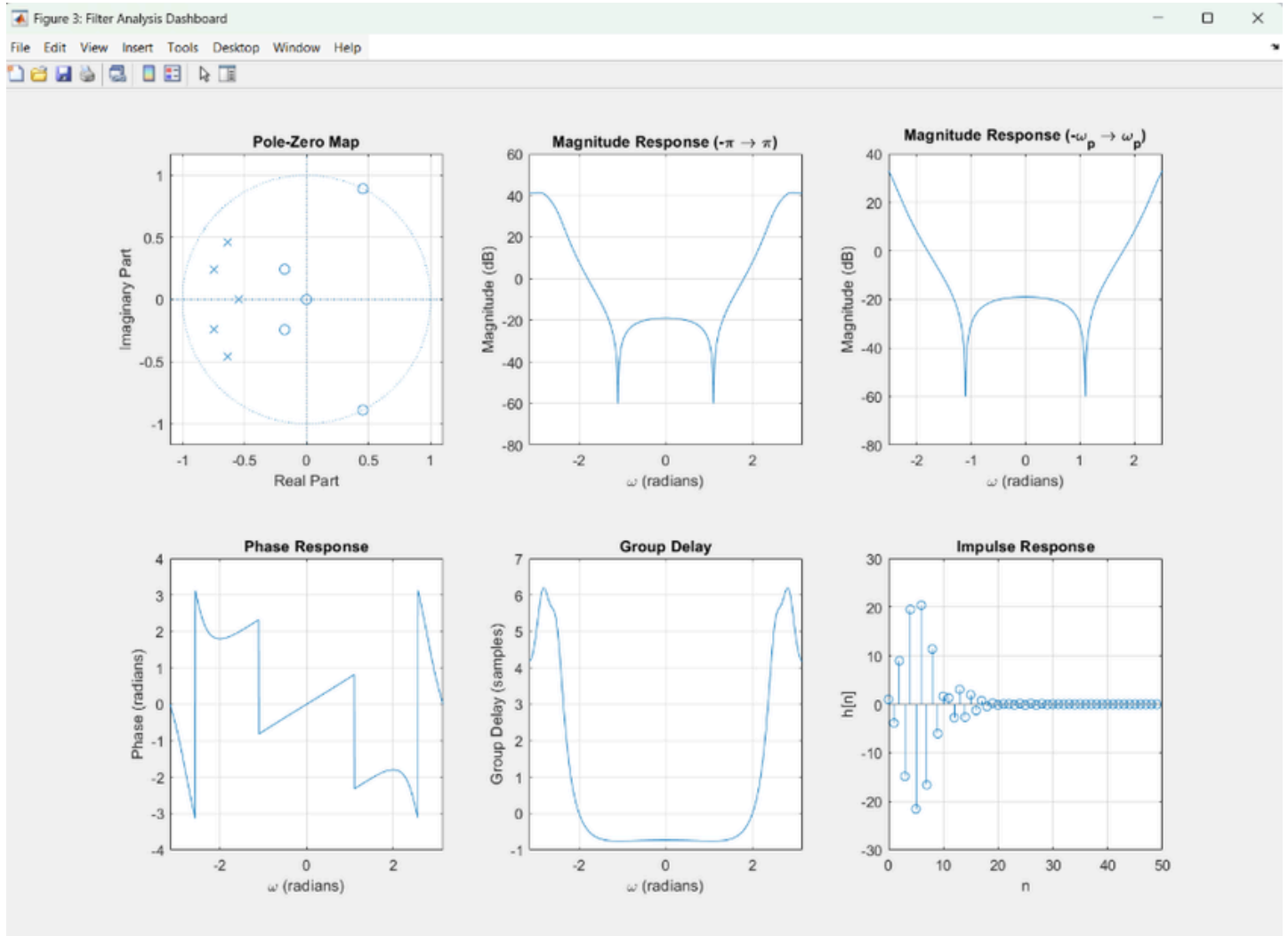
After obtaining the new high-pass pole-zero locations, the zp2tf function was used to convert the zero-pole-gain representation back into transfer function coefficients.

Ripple

```
93      %HPF Ripples
94      W_hpf = linspace(pi - wp2, pi+wp2, 1024);
95      H_hpf = freqz(num_hpf, den_hpf, W_hpf);
96      Mag_dB_hpf = 20*log10(abs(H_hpf));
97      Ripples_hpf = max(Mag_dB_hpf) - min(Mag_dB_hpf); |
98      fprintf('\n--- HPF Analysis ---\n');
99      fprintf('Passband ripples for HPF: %.3f dB\n', Ripples_hpf);
100
101      system_analysis(num_hpf, den_hpf, pi - wp);
```

Passband ripples for HPF: 0.412 dB

Plots:



These plots show the analysis of the high-pass filter. From the pole-zero map, all poles remain inside the unit circle after the frequency transformation, confirming that the filter is stable. Compared to the 5th-order low-pass filter, the pole and zero locations are rotated around the unit circle, which changes the frequency-selective behavior while preserving stability. The most noticeable difference appears in the magnitude response, where the high gain that originally existed around $\omega=0$ is shifted toward the high-frequency region around $\omega=\pm\pi$, while the low-frequency components are now attenuated. Unlike the low-pass filter, whose passband is centered at DC, the high-pass filter rejects low frequencies and passes the high-frequency components with nearly constant gain and only slight ripple. The phase response changes significantly because of the pole-zero rotation, leading to a different phase distribution and discontinuities across the frequency range. Similarly, the group delay profile is shifted, with its largest values now occurring around the new passband region instead of the locations observed in the low-pass filter. The impulse response remains bounded and decays with time, confirming that the transformed system is stable, although its oscillation pattern differs slightly due to the modified pole-zero configuration. Overall, the frequency transformation successfully converts the original low-pass filter into a high-pass filter while preserving stability.

- BPF Centered at $\pi/2$

```

105     z_bpf = [z_lpf * exp( 1j * pi/2);
106             z_lpf * exp(-1j * pi/2)];
107
108     p_bpf = [p_lpf * exp( 1j * pi/2);
109             p_lpf * exp(-1j * pi/2)];
110

```

Rotating the pole-zero pattern by $\pi/2$ means the complex poles and zeros no longer appear in conjugate pairs, which results in complex coefficients.

To guarantee real coefficients, we must restore conjugate symmetry by rotating by both $+\pi/2$ and $-\pi/2$. This ensures that for every pole/zero at angle θ , there is a corresponding conjugate at angle $-\theta$, which is the necessary and sufficient condition for a digital filter to have real-valued coefficients.

Ripple

```

117 %BPF Ripples
118 W_bpf = linspace(pi/2 - wp2, pi/2 +wp2, 1024);
119 H_bpf = freqz(num_bpf, den_bpf, W_bpf);
120 Mag_dB_bpf = 20*log10(abs(H_bpf));
121 Ripples_bpf = max(Mag_dB_bpf) - min(Mag_dB_bpf);
122 fprintf('Passband ripples for BPF: %.3f dB\n', Ripples_bpf);

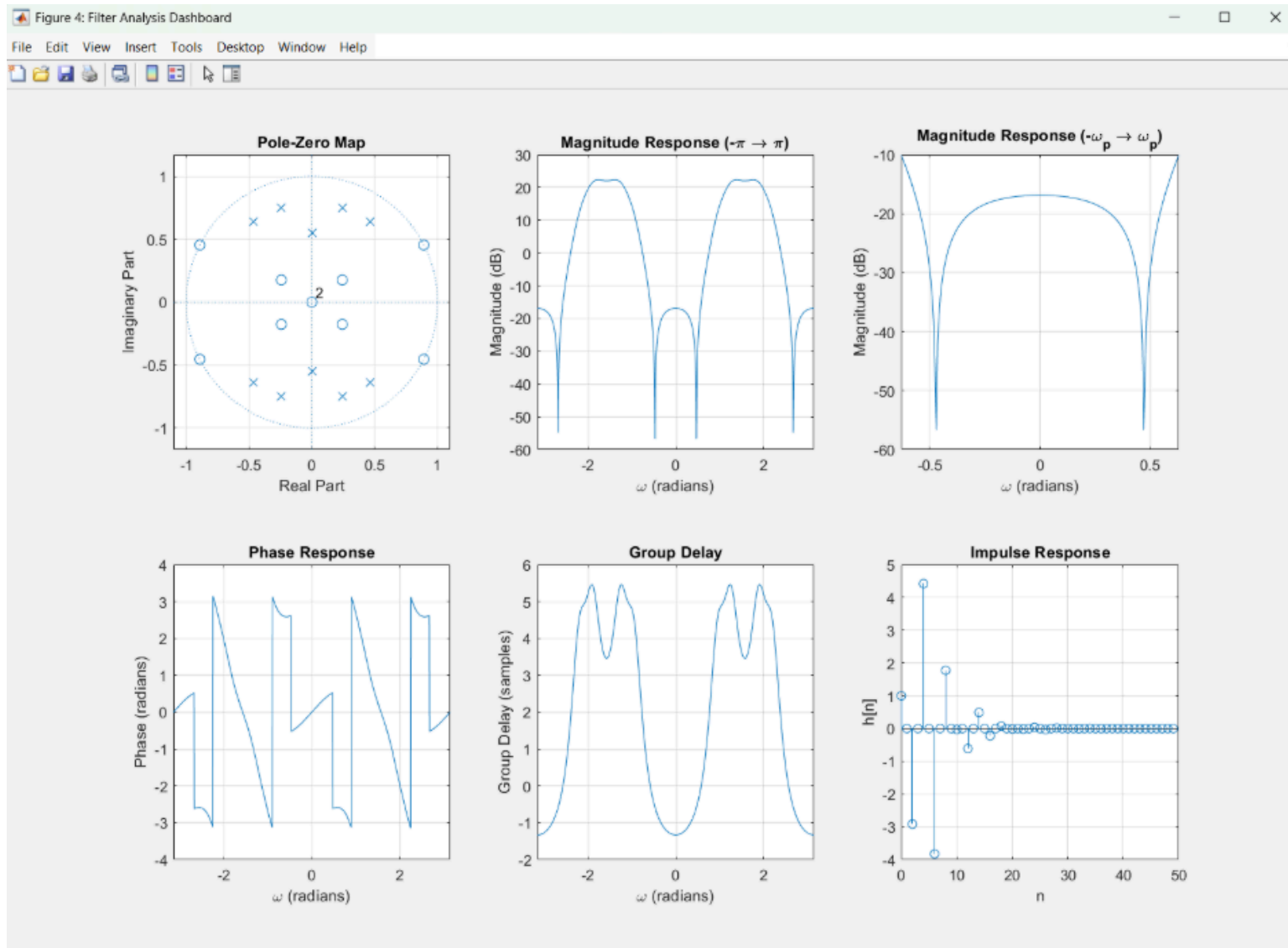
```

```

Passband ripples for BPF: 0.653 dB

```

plots:



These plots show the analysis of the transformed band-pass filter obtained from the original 5th-order low-pass filter using frequency transformation. Instead of simply rotating all poles and zeros by $(+\pi/2)$, the transformation duplicates every pole and zero and places one copy at $(+\pi/2)$ and the other at $(-\pi/2)$. This symmetric placement is essential because it preserves the conjugate symmetry of the pole-zero locations, ensuring that the resulting filter has real-valued coefficients and a physically realizable implementation. From the pole-zero map, all poles remain inside the unit circle even after the transformation, confirming that the new filter is stable. Compared to the original low-pass filter, the poles and zeros are no longer concentrated around a single frequency region but are distributed symmetrically about the real axis, producing two identical frequency bands. This symmetric arrangement is directly responsible for the characteristic band-pass behavior observed in the magnitude response. As it shows that the single low-frequency passband of the original filter has been transformed into two symmetric passbands centered approximately around $(\omega = \pi/2)$. At the same time, frequencies around DC and near the band edges are strongly attenuated. Although the filter order has increased, the passband remains relatively flat with only small ripples, indicating that the desired frequency characteristics are successfully preserved after the transformation.

Section 8: Comb Filter Analysis

In Section 8, a comb filter was designed using the optimized 5th-order lowpass filter as a foundation. Based on the time-domain definition, the Z-domain transfer function evaluates to $H_{\text{comp}}(z) = H(z^8)$. Physically, this mathematical substitution compresses the original filter's frequency response. Because the frequency variable is scaled by a factor of 8, the filter sweeps through its spectral shape eight times faster. This causes the original passband to shrink to one-eighth of its original width (from $\pm\omega_p$ down to $\pm\omega_p/8$). Since digital frequencies are periodic every 2π radians, this newly compressed shape loops eight times across the standard frequency spectrum. This repetition creates the alternating stopbands and passbands of the comb filter. In MATLAB, this transformation was achieved by zero-padding the original 5th-order filter coefficients, inserting L-1 zeros between each existing term to successfully simulate the z^8 scaling.

$$\begin{aligned}
 h_{\text{comp}}[n] &= \sum_{m=-\infty}^{\infty} h[m] \delta[n-8m] \\
 H_{\text{comp}}(z) &= \sum_{n=-\infty}^{\infty} h_{\text{comp}}[n] z^{-n} = \sum_{n=-\infty}^{\infty} \left(\sum_{m=-\infty}^{\infty} h[m] \delta[n-8m] \right) z^{-n} \\
 &= \sum_{m=-\infty}^{\infty} h[m] \sum_{n=-\infty}^{\infty} \delta[n-8m] z^{-n} \\
 &= \sum_{m=-\infty}^{\infty} h[m] z^{-8m} = \sum_{m=-\infty}^{\infty} h[m] (z^8)^{-m} \\
 H_{\text{comp}}(z) &= H(z^8) \xrightarrow{z=e^{j\omega}} H_{\text{comp}}(e^{j8\omega}) = H(e^{j8\omega}) \\
 H_{\text{comp}}(e^{j(\omega + \frac{2\pi}{8})}) &= H(e^{j(\omega + \frac{2\pi}{8}) \cdot 8}) = H(e^{j(\omega + \frac{2\pi}{8}) \cdot 8}) \\
 \text{test for periodicity} \\
 H_{\text{comp}}(e^{j(\omega + \frac{2\pi}{8})}) &= H(e^{j\omega})
 \end{aligned}$$

%% Section 8: Comb Filter

```
L = 8;
num_5th = poly([zero1 zero2 zero3 zero4]);
den_5th = poly([optimal_r2 pole2 pole3 pole4 pole5]);

len_num = (length(num_5th) - 1) * L + 1;
len_den = (length(den_5th) - 1) * L + 1;

num_comb = zeros(1, len_num);
den_comb = zeros(1, len_den);

num_comb(1:L:end) = num_5th;
den_comb(1:L:end) = den_5th;

system_analysis(num_comb, den_comb, wp);

W_comb = linspace(-wp2/L, wp2/L, 1024); % The passband is compressed by L
H_comb = freqz(num_comb, den_comb, W_comb);
Mag_dB_comb = 20*log10(abs(H_comb));
Ripples_comb = range(Mag_dB_comb);

fprintf('Comb Filter passband ripples: %.3f dB\n', Ripples_comb);
```

minimum ripples: 0.648 dB

optimal r: 0.787

Passband ripples before adjusting magnitude of p1: 0.577 dB

minimum ripples: 0.412 dB

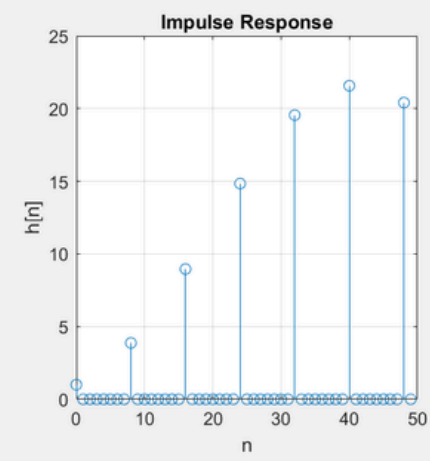
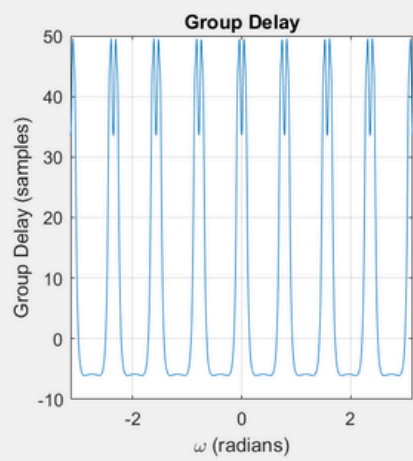
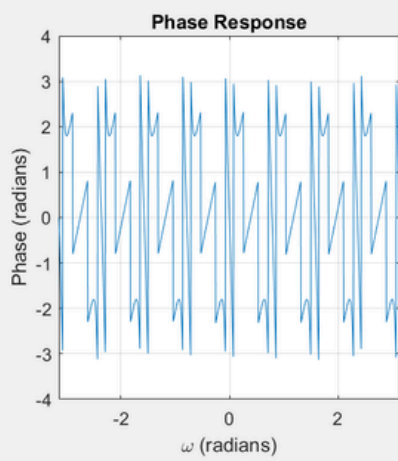
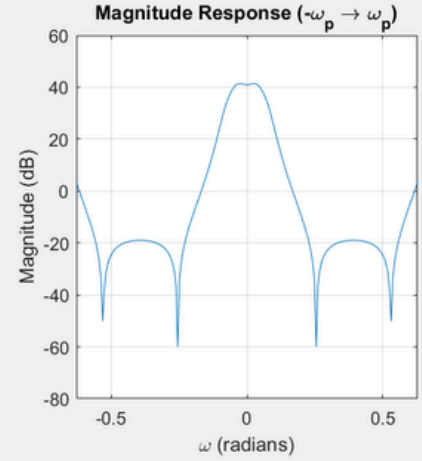
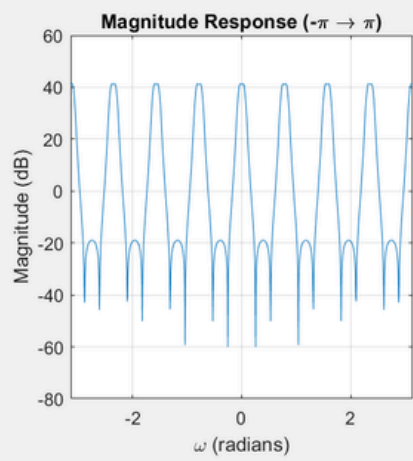
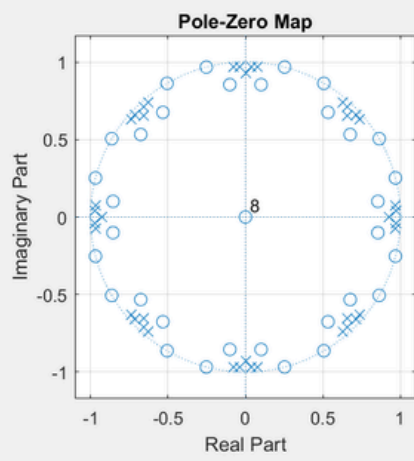
optimal r2: 0.548

--- HPF Analysis ---

Passband ripples for HPF: 0.412 dB

Passband ripples for BPF: 0.653 dB

Comb Filter passband ripples: 0.412 dB



```

function system_analysis(num, den, wp)
    w = linspace(-pi, pi, 1024);
    figure('Name', 'Filter Analysis Dashboard', 'Position', [100, 100, 1200, 800]);
    H = freqz(num, den, w);
    mag_dB = 20*log10(abs(H));
    % 1. Pole-Zero Plot
    subplot(2, 3, 1);
    zplane(num, den);
    grid on;
    title("Pole-Zero Map");
    % 2. Full Magnitude Response (-pi to pi)
    subplot(2, 3, 2);
    plot(w, mag_dB);
    grid on;
    xlim([-pi, pi]);
    xlabel("\omega (radians)");
    ylabel("Magnitude (dB)");
    title('Magnitude Response (-\pi \rightarrow \pi)');
    % 3. Passband Magnitude Response (-wp to wp) to check ripples
    subplot(2, 3, 3);
    plot(w, mag_dB);
    grid on;
    xlim([-wp, wp]);
    xlabel("\omega (radians)");
    ylabel("Magnitude (dB)");
    title('Magnitude Response (-\omega_p \rightarrow \omega_p)');
    % 4. Phase Response
    subplot(2, 3, 4);
    phi = angle(H);
    plot(w, phi);
    grid on;
    xlim([-pi, pi]);
    xlabel("\omega (radians)");
    ylabel("Phase (radians)");
    title('Phase Response');
    % 5. Group Delay
    subplot(2, 3, 5);
    gd = grpdelay(num, den, w);
    plot(w, gd);
    grid on;
    xlim([-pi, pi]);
    xlabel('\omega (radians)');
    ylabel('Group Delay (samples)');
    title('Group Delay');
    % 6. Impulse Response
    subplot(2, 3, 6);
    [h, t] = impz(num, den, 50);
    stem(t, h);
    grid on;
    xlabel('n');
    ylabel('h[n]');
    title('Impulse Response');
end

```

```
%% plotting zeros and poles of transfer function
```

```
A= -0.6;  
a = 1;  
b = [1 -A];  
subplot(3,2,1)  
zplane(a,b);  
title('Z-plane with Poles and Zeros');
```

```
%% Magnitude response in dB in freq range from -pi to pi
```

```
w = -pi:pi/100:pi;  
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);  
mag_dB = 20*log10(mag);  
subplot(3,2,2)  
grid on;  
plot (w,mag_dB);  
xlabel ('w');  
ylabel ('mag_ dB');  
title('Magnitude response in dB');
```

```
%% Magnitude response in dB in freq range from wp to pi & evaluating the ripples
```

```
w = 2.91:pi/1000:pi;  
mag = 1./sqrt ((1-A*cos(w)).^2+(A*sin(w)).^2);  
mag_dB = 20*log10(mag);  
subplot(3,2,3)  
grid on;  
plot (w,mag_dB);  
xlabel ('w');  
ylabel ('mag_ dB');  
title('Magnitude response in dB in freq range from wp to pi');
```

```
%% Phase response
```

```
w = -pi:pi/100:pi;  
real_part = (1 - A*cos(w));  
imag_part = (A*sin(w));  
phase = -atan2(imag_part, real_part);  
subplot(3,2,4)  
plot(w, phase);  
grid on;  
xlabel('\omega');  
ylabel('Phase (radians)');  
title('Phase Response');
```

```

% numerical derivative
dphi = diff(phase);
dw = diff(w);
gd = -dphi ./ dw;
% plotting
subplot(3,2,5)
plot(w(1:end-1), gd);
grid on;
xlabel('\omega');
ylabel('Group Delay');
title('Group Delay');


---


%% calculation of inverse z-transform
syms z n
H_z = 1/(1 - A*z^(-1));
h_n = iztrans(H_z, z, n);


---


%% Impulse response plot
n_plot = 0:36;
h_plot = double(subs(h_n, n, n_plot));
subplot(3,2,6)
stem(n_plot, h_plot, 'filled');
grid on;
xlabel('n');
ylabel('h[n]');
title('Impulse Response');

pole1=0.6;
wc = 0.25 * pi;           % Cutoff frequency
width = 0.1 * pi;         % Transition band width
ws = wc + (width / 2);    % Stopband edge
wp = wc - (width / 2);    % Passband edge

%%% zeros
zero1 = 0.3*exp(j*ws);
zero2 = 0.3*exp(-j*ws);
zeros_poly = poly([zero1 zero2]);
W = linspace(-(wp), wp, 1024);
Ripples = 1; %%dB
Counter = 0;

```

```

for r = 0 : 0.001 : 1
    Counter = Counter + 1;
    pole2 = r*exp(j*wp);
    pole3 = r*exp(-j*wp);
    poles = poly([pole1 pole2 pole3]);

    H = freqz(zeros_poly,poles,W);
    Mag_dB = 20*log10(abs(H));
    mag(Counter) = r;
    Storage(Counter) = max(Mag_dB) - min(Mag_dB);
end

[minR,i] = min(Storage);
optimal_r = mag(i);
fprintf('minimum ripples: %.3f dB\n', minR);
fprintf('optimal r: %.3f\n', optimal_r);

% plotting
num = poly([zero1 zero2]);
den = poly([pole1 optimal_r*exp(1j*wp) optimal_r*exp(-1j*wp)]);
system_analysis(num, den, wp);

%%% Section 6: Fifth Order Lowpass Filter
wp2 = wp/2;
ws2 = (pi + ws) / 2;

pole2 = optimal_r * exp(j * wp);
pole3 = optimal_r * exp(-j * wp);
pole4 = optimal_r * exp(j * wp2);
pole5 = optimal_r * exp(-j * wp2);
poles = poly([pole1 pole2 pole3 pole4 pole5]);

zero3 = exp(j * ws2);
zero4 = exp(-j * ws2);
zeros_poly = poly([zero1 zero2 zero3 zero4]);

W = linspace(-wp2, wp2, 1024);
H = freqz(zeros_poly, poles, W);
Mag_dB = 20*log10(abs(H));
Ripples = range(Mag_dB);
fprintf("Passband ripples before adjusting magnitude of p1: %.3f dB\n", Ripples);

```



```

Counter = 0;
for r = 0 : 0.001 : 1
    Counter = Counter + 1;
    pole1 = r;
    poles = poly([pole1 pole2 pole3 pole4 pole5]);
    H = freqz(zeros_poly, poles, W);
    Mag_dB = 20*log10(abs(H));
    mag2(Counter) = r;
    Storage2(Counter) = max(Mag_dB) - min(Mag_dB);
end

[minR,i] = min(Storage2);
optimal_r2 = mag2(i);
fprintf('minimum ripples: %.3f dB\n', minR);
fprintf('optimal r2: %.3f\n', optimal_r2);

% plotting
num = poly([zero1 zero2 zero3 zero4]);
den = poly([optimal_r2 optimal_r*exp(1j*wp) optimal_r*exp(-1j*wp) optimal_r*exp(1j * wp2) optimal_r*exp(-1j * wp2)]);
system_analysis(num, den, wp2);

%HPF
[z_lpf, p_lpf, k_lpf] = tf2zpk(num, den);

z_hpf = z_lpf * exp(1j * pi); %rotating by  $\pi$ 
p_hpf = p_lpf * exp(1j * pi);

[num_hpf, den_hpf] = zp2tf(z_hpf, p_hpf, k_lpf);
num_hpf = real(num_hpf);
den_hpf = real(den_hpf);

%HPF Ripples
W_hpf = linspace(pi - wp2, pi+wp2, 1024);
H_hpf = freqz(num_hpf, den_hpf, W_hpf);
Mag_dB_hpf = 20*log10(abs(H_hpf));
Ripples_hpf = max(Mag_dB_hpf) - min(Mag_dB_hpf);
fprintf('\n--- HPF Analysis ---\n');
fprintf('Passband ripples for HPF: %.3f dB\n', Ripples_hpf);

system_analysis(num_hpf, den_hpf, pi - wp);

% BPF centered at  $\pi/2$ 
z_bpf = [z_lpf * exp( 1j * pi/2); %we rotate by both  $+\pi/2$  and  $-\pi/2$  to restore symmetry (real coefficients)
         z_lpf * exp(-1j * pi/2)];

p_bpf = [p_lpf * exp( 1j * pi/2);
         p_lpf * exp(-1j * pi/2)];

k_bpf = k_lpf^2;

[num_bpf, den_bpf] = zp2tf(z_bpf, p_bpf, k_bpf);
num_bpf = real(num_bpf);
den_bpf = real(den_bpf);

%BPF Ripples
W_bpf = linspace(pi/2 - wp2, pi/2 +wp2, 1024);
H_bpf = freqz(num_bpf, den_bpf, W_bpf);
Mag_dB_bpf = 20*log10(abs(H_bpf));
Ripples_bpf = max(Mag_dB_bpf) - min(Mag_dB_bpf);
fprintf('Passband ripples for BPF: %.3f dB\n', Ripples_bpf);

system_analysis(num_bpf, den_bpf, wp);

```

```
%% Section 8: Comb Filter
```

```
L = 8;  
num_5th = poly([zero1 zero2 zero3 zero4]);  
den_5th = poly([optimal_r2 pole2 pole3 pole4 pole5]);  
  
len_num = (length(num_5th) - 1) * L + 1;  
len_den = (length(den_5th) - 1) * L + 1;  
  
num_comb = zeros(1, len_num);  
den_comb = zeros(1, len_den);  
  
num_comb(1:L:end) = num_5th;  
den_comb(1:L:end) = den_5th;  
  
system_analysis(num_comb, den_comb, wp);  
  
W_comb = linspace(-wp2/L, wp2/L, 1024); % The passband is compressed by L  
H_comb = freqz(num_comb, den_comb, W_comb);  
Mag_dB_comb = 20*log10(abs(H_comb));  
Ripples_comb = range(Mag_dB_comb);  
  
fprintf('Comb Filter passband ripples: %.3f dB\n', Ripples_comb);
```

```
minimum ripples: 0.648 dB
```

```
optimal r: 0.787
```

```
Passband ripples before adjusting magnitude of p1: 0.577 dB
```

```
minimum ripples: 0.412 dB
```

```
optimal r2: 0.548
```

```
--- HPF Analysis ---
```

```
Passband ripples for HPF: 0.412 dB
```

```
Passband ripples for BPF: 0.653 dB
```

```
Comb Filter passband ripples: 0.412 dB
```